Projet de classification de feuilles d’arbres

Compte rendu final

Partie 1 : classification avec 4 types de feuilles

Partie 2 : classification avec 32 types de feuilles

Méthodes d’apprentissage – ISN 2A

Fait par : **MEHDAOUI Hafsa**

7 mai 2026

Table des matières

Introduction générale **2**

1 Classification avec 4 types de feuilles **3**

1.1 Introduction 3

1.2 Création de la base de données 3

1.3 Architecture utilisée 3

1.4 Étude des paramètres 4

1.4.1 Influence du nombre d'epochs 4

1.4.2 Influence de la profondeur du réseau 4

1.4.3 Influence du dropout 5

1.4.4 Influence de l'optimiseur 5

1.5 Choix du modèle final pour la base à 4 classes 5

1.6 Conclusion de la première partie 6

2 Classification avec 32 types de feuilles **7**

2.1 Introduction 7

2.2 Création de la base de données 7

2.3 Architecture utilisée 7

2.4 Étude des paramètres 8

2.4.1 Influence du nombre d'epochs 8

2.4.2 Influence de la profondeur du réseau 9

2.4.3 Influence du dropout 9

2.4.4 Influence de l'optimiseur 9

2.5 Choix du modèle final pour la base à 32 classes 10

2.6 Conclusion de la deuxième partie 10

Conclusion générale **11**

Introduction générale

L'objectif de ce projet est de réaliser une classification automatique de feuilles d'arbres à l'aide d'un réseau de neurones convolutionnel. Le travail est divisé en deux parties. Dans la première partie, on utilise une base réduite contenant 4 types de feuilles. Dans la deuxième partie, on utilise la base complète contenant 32 types de feuilles.

Le but est de trouver une architecture adaptée pour chaque base de données, en obtenant une précision supérieure à 75%. Pour cela, plusieurs expériences sont réalisées en modifiant certains paramètres du modèle, comme le nombre d'epochs, la profondeur du réseau, l'utilisation du dropout et l'optimiseur.

Afin de comparer les deux parties de manière cohérente, les mêmes expériences sont gardées dans la première et dans la deuxième partie. Le seul élément qui change est la base de données utilisée. Cette démarche permet de montrer qu'une architecture qui fonctionne très bien sur une base simple ne donne pas forcément les mêmes performances sur une base plus complexe.

1 Classification avec 4 types de feuilles

1.1 Introduction

Dans cette première partie, on s'intéresse uniquement à la classification de 4 classes : `bitter orange`, `rose`, `sweet potato` et `vieux garcon`.

Le but est de construire une architecture capable de reconnaître correctement ces différentes feuilles à partir des images fournies. Pour cela, plusieurs architectures et plusieurs paramètres sont testés afin d'obtenir une précision supérieure à 75%, tout en évitant d'utiliser un modèle trop complexe si une architecture plus simple donne déjà de bons résultats.

1.2 Création de la base de données

Pour cette première partie, la base utilisée contient 620 images appartenant à 4 classes différentes. Les images sont chargées avec la fonction `image_dataset_from_directory` de Keras. Cette fonction permet de créer automatiquement une base de données à partir d'un dossier organisé en sous-dossiers, chaque sous-dossier correspondant à une classe.

La base est divisée en deux parties : une partie pour l'apprentissage et une partie pour la validation. Dans mon programme, 496 images sont utilisées pour l'apprentissage et 124 images sont utilisées pour la validation. Ensuite, la partie validation est séparée en deux afin d'obtenir également un jeu de test. Ce jeu de test permet d'évaluer le modèle final sur des images qui n'ont pas été utilisées pendant l'apprentissage.

Les images sont redimensionnées en taille 128×128 pixels. Les données sont ensuite préparées avec des opérations comme le mélange des données, la mise en cache et le préchargement, afin de rendre l'entraînement plus efficace.

1.3 Architecture utilisée

Pour cette classification, j'ai utilisé un réseau de neurones convolutionnel, car ce type de modèle est adapté au traitement des images. Le réseau commence par une étape d'augmentation de données avec des transformations simples comme le retournement horizontal, une légère rotation et un zoom. Cette étape permet de rendre le modèle plus robuste, car elle lui apprend à reconnaître une feuille même si son orientation ou sa taille varie légèrement.

Les pixels des images sont ensuite normalisés avec une couche `Rescaling(1.0 / 255)`, afin que les valeurs soient comprises entre 0 et 1. Cette normalisation facilite l'apprentissage du modèle.

J'ai testé plusieurs variantes d'architectures :

Variante	Description
A	Deux blocs convolutionnels
B	Trois blocs convolutionnels
C	Trois blocs convolutionnels avec dropout

Table 1 – Variantes d'architectures testées pour la base à 4 classes

Dans chaque bloc convolutionnel, on applique une couche de convolution `Conv2D`, suivie d'une couche de réduction `MaxPooling2D`. Les convolutions permettent d'extraire les caractéristiques importantes des images, comme les formes et les contours des feuilles. Le max pooling permet de réduire la taille des données tout en conservant les informations principales.

Après les blocs convolutionnels, les données sont aplaties avec `Flatten`, puis envoyées vers une couche dense de 128 neurones. La dernière couche contient 4 neurones avec une activation `softmax`, car il y a 4 classes à prédire.

1.4 Étude des paramètres

Pour obtenir une architecture satisfaisante, j'ai réalisé plusieurs expériences en modifiant certains paramètres. L'objectif était de comparer les performances des modèles en observant la précision d'entraînement, la précision de validation, la précision de test et la perte.

Les expériences réalisées sont les suivantes :

Expérience	Var.	Optim.	Epochs	Best val. acc.	Test acc.
P1_ep3_A_adam	A	Adam	3	0.9833	1.0000
P1_ep5_A_adam	A	Adam	5	0.9833	1.0000
P1_ep10_A_adam	A	Adam	10	1.0000	0.9844
P1_2blocs_A_adam	A	Adam	5	1.0000	1.0000
P1_3blocs_B_adam	B	Adam	5	0.9833	0.9844
P1_3blocs_C_dropout03	C	Adam	5	1.0000	1.0000
P1_A_rmsprop	A	RMSprop	5	0.9833	1.0000

Table 2 – Résultats des expériences pour la base à 4 classes

1.4.1 Influence du nombre d'epochs

Dans un premier temps, j'ai testé l'influence du nombre d'epochs avec la même architecture simple, la variante A. Avec seulement 3 epochs, le modèle atteint déjà une très bonne précision de validation de 0.9833 et une précision de test de 1.0000. En augmentant à 5 epochs, les résultats restent excellents. Avec 10 epochs, la précision de validation atteint 1.0000, mais la précision de test est légèrement plus basse, avec 0.9844.

Cela montre que le modèle apprend très rapidement sur la base à 4 classes. Comme la base contient peu de classes, une architecture simple est suffisante pour obtenir une précision largement supérieure à 75%.

1.4.2 Influence de la profondeur du réseau

J'ai ensuite comparé une architecture à deux blocs convolutionnels avec une architecture à trois blocs convolutionnels. La variante A, qui contient deux blocs convolutionnels, donne déjà une précision de validation de 1.0000 et une précision de test de 1.0000. La variante B, plus profonde, donne une précision de validation de 0.9833 et une précision de test de 0.9844.

L'ajout d'un troisième bloc convolutionnel n'améliore donc pas les résultats dans cette première partie. Au contraire, l'architecture plus simple obtient des résultats au moins

aussi bons. Cela montre qu'il n'est pas nécessaire d'utiliser un modèle plus complexe pour cette base à 4 classes.

1.4.3 Influence du dropout

J'ai aussi testé une architecture avec dropout, la variante C. Le dropout sert à limiter le surapprentissage en désactivant aléatoirement certains neurones pendant l'entraînement. Cette architecture obtient également une précision de validation de 1.0000 et une précision de test de 1.0000.

Cependant, comme l'architecture simple sans dropout donne déjà les mêmes résultats, l'ajout du dropout n'est pas indispensable pour cette première base. Il pourra être plus utile dans la deuxième partie, avec 32 classes, où le problème sera plus complexe.

1.4.4 Influence de l'optimiseur

Enfin, j'ai comparé l'optimiseur Adam avec RMSprop. Avec l'architecture A, RMSprop donne une précision de validation de 0.9833 et une précision de test de 1.0000. Les résultats sont donc très bons, mais Adam donne des résultats au moins aussi bons avec l'architecture choisie.

J'ai donc conservé Adam pour le modèle final, car il donne de très bonnes performances avec une architecture simple.

1.5 Choix du modèle final pour la base à 4 classes

Pour la base à 4 classes, j'ai choisi le modèle `P1_2blocs_A_adam`. Ce modèle utilise l'architecture A, c'est-à-dire deux blocs convolutionnels, avec l'optimiseur Adam, un taux d'apprentissage de 0.001, une taille de batch de 32 et 5 epochs.

Ce choix est justifié par le fait que le modèle atteint une précision de validation de 1.0000 et une précision de test de 1.0000. Il dépasse donc largement l'objectif de 75%. De plus, il reste plus simple que les architectures B et C, qui ajoutent un troisième bloc convolutionnel ou du dropout sans apporter d'amélioration significative.

Le modèle final retenu pour cette première partie est donc :

Paramètre	**Valeur**
Architecture	Variante A
Nombre de blocs convolutionnels	2
Optimiseur	Adam
Learning rate	0.001
Batch size	32
Epochs	5
Dropout	0.0
Best val accuracy	1.0000
Test accuracy	1.0000

Table 3 – Architecture finale retenue pour la base à 4 classes

1.6 Conclusion de la première partie

Pour conclure cette première partie, la classification des 4 types de feuilles donne de très bons résultats. Les différentes expériences montrent que la base est relativement simple à apprendre pour le réseau de neurones convolutionnel. Une architecture simple avec deux blocs convolutionnels suffit pour atteindre une précision supérieure à 75%.

Les architectures plus complexes, comme le modèle à trois blocs ou le modèle avec dropout, ne sont pas nécessaires dans cette première partie, car elles n'améliorent pas réellement les performances. Le modèle retenu est donc `P1_2blocs_A_adam`, car il combine de très bons résultats avec une architecture simple. Pour la deuxième partie, avec 32 classes, il faudra comparer les mêmes expériences afin de voir si elles restent efficaces sur une base plus complexe.

2 Classification avec 32 types de feuilles

2.1 Introduction

Dans cette deuxième partie, on s'intéresse à la classification de la base complète contenant 32 types de feuilles d'arbres. L'objectif reste le même que dans la première partie : construire un réseau de neurones convolutionnel capable de reconnaître correctement les feuilles et d'obtenir une précision supérieure à 75%.

Pour garder une étude cohérente, j'ai conservé les mêmes expériences que dans la première partie. Le seul élément qui change est la base de données utilisée : on passe d'une base réduite de 4 classes à une base complète de 32 classes. Cette méthode permet de comparer les deux situations et de voir si les architectures qui fonctionnent bien sur la première base restent efficaces lorsque le problème devient plus complexe.

2.2 Création de la base de données

Pour cette deuxième partie, la base utilisée contient 4955 images appartenant à 32 classes différentes. Les images sont chargées avec la fonction `image_dataset_from_directory` de Keras, comme dans la première partie. Cette fonction permet de créer automatiquement une base de données à partir d'un dossier organisé en sous-dossiers, chaque sous-dossier correspondant à une classe.

La base est divisée en deux parties : une partie pour l'apprentissage et une partie pour la validation. Dans mon programme, 3964 images sont utilisées pour l'apprentissage et 991 images sont utilisées pour la validation. Ensuite, la partie validation est séparée en deux afin d'obtenir également un jeu de test. Ce jeu de test permet d'évaluer le modèle final sur des images qui n'ont pas été utilisées pendant l'apprentissage.

Les images sont redimensionnées en taille 128×128 pixels. Les données sont ensuite préparées avec les mêmes opérations que dans la première partie : mélange des données, mise en cache et préchargement. Cela permet de garder les mêmes conditions d'entraînement pour comparer les résultats des deux bases.

Cette base est plus difficile que la première, car le modèle doit distinguer 32 classes au lieu de seulement 4. Certaines feuilles peuvent aussi avoir des formes ou des couleurs proches, ce qui rend la classification plus complexe.

2.3 Architecture utilisée

Pour cette classification, j'ai utilisé le même type de modèle que dans la première partie : un réseau de neurones convolutionnel. Ce type de modèle est adapté au traitement des images, car les couches convolutionnelles permettent d'extraire des informations importantes comme les contours, les formes et certaines textures des feuilles.

Le réseau commence par une étape d'augmentation de données avec des transformations simples comme le retournement horizontal, une légère rotation et un zoom. Cette étape permet de rendre le modèle plus robuste, car elle lui apprend à reconnaître une feuille même si son orientation ou sa taille varie légèrement.

Les pixels des images sont ensuite normalisés avec une couche `Rescaling(1.0 / 255)`, afin que les valeurs soient comprises entre 0 et 1. Cette normalisation facilite l'apprentissage du modèle.

J'ai testé les mêmes variantes d'architectures que dans la première partie :

Variante	Description
A	Deux blocs convolutionnels
B	Trois blocs convolutionnels
C	Trois blocs convolutionnels avec dropout

Table 4 – Variantes d'architectures testées pour la base à 32 classes

Dans chaque bloc convolutionnel, on applique une couche de convolution `Conv2D`, suivie d'une couche de réduction `MaxPooling2D`. Les convolutions permettent d'extraire les caractéristiques importantes des images. Le max pooling permet de réduire la taille des données tout en conservant les informations principales.

Après les blocs convolutionnels, les données sont aplaties avec `Flatten`, puis envoyées vers une couche dense de 128 neurones. La dernière couche contient 32 neurones avec une activation `softmax`, car il y a maintenant 32 classes à prédire.

2.4 Étude des paramètres

Pour obtenir une architecture satisfaisante, j'ai réalisé les mêmes expériences que dans la première partie. L'objectif est de comparer les performances des modèles en observant la précision d'entraînement, la précision de validation, la précision de test et la perte.

Les expériences réalisées sont les suivantes. Dans le code, certains noms commencent encore par `P1`, mais dans le compte rendu je les note avec `P2`, car ils correspondent à la deuxième partie.

Expérience	Var.	Optim.	Epochs	Best val. acc.	Test acc.
`P2_ep3_A_adam`	A	Adam	3	0.6223	0.6625
`P2_ep5_A_adam`	A	Adam	5	0.6928	0.6938
`P2_ep10_A_adam`	A	Adam	10	0.8415	0.8250
`P2_2blocs_A_adam`	A	Adam	5	0.7202	0.7417
`P2_3blocs_B_adam`	B	Adam	5	0.6732	0.6750
`P2_3blocs_C_dropout03`	C	Adam	5	0.8004	0.7604
`P2_A_rmsprop`	A	RMSprop	5	0.7045	0.7271

Table 5 – Résultats des expériences pour la base à 32 classes

2.4.1 Influence du nombre d'epochs

Dans un premier temps, j'ai testé l'influence du nombre d'epochs avec la même architecture simple, la variante A. Avec 3 epochs, le modèle atteint une précision de validation de 0.6223 et une précision de test de 0.6625. Ces résultats sont inférieurs à l'objectif de 75%.

En augmentant à 5 epochs, la précision s'améliore légèrement : la précision de validation atteint 0.6928 et la précision de test atteint 0.6938. Cependant, ces valeurs restent encore en dessous de 75%.

Avec 10 epochs, le modèle atteint une précision de validation de 0.8415 et une précision de test de 0.8250. Cette fois, l'objectif est dépassé. Cela montre que la base à 32 classes nécessite un apprentissage plus long que la base à 4 classes. Dans la première partie, quelques epochs suffisaient déjà pour obtenir une très bonne précision, alors qu'ici le modèle a besoin de plus de passages sur les données.

2.4.2 Influence de la profondeur du réseau

J'ai ensuite comparé une architecture à deux blocs convolutionnels avec une architecture à trois blocs convolutionnels. La variante A avec 5 epochs donne une précision de validation de 0.7202 et une précision de test de 0.7417. Cette valeur est proche de l'objectif de 75%, mais elle reste légèrement insuffisante.

La variante B, plus profonde, donne une précision de validation de 0.6732 et une précision de test de 0.6750. L'ajout d'un troisième bloc convolutionnel n'améliore donc pas les résultats dans cette expérience. Au contraire, le modèle plus profond obtient une précision plus faible que la variante A.

Cela montre qu'augmenter la profondeur du réseau ne garantit pas automatiquement une meilleure classification. Pour cette base, le nombre d'epochs a eu un effet plus important que l'ajout d'un bloc convolutionnel supplémentaire.

2.4.3 Influence du dropout

J'ai aussi testé une architecture avec dropout, la variante C. Le dropout sert à limiter le surapprentissage en désactivant aléatoirement certains neurones pendant l'entraînement. Cette architecture obtient une précision de validation de 0.8004 et une précision de test de 0.7604.

Le résultat dépasse l'objectif de 75%, mais il reste inférieur au meilleur modèle obtenu avec la variante A sur 10 epochs. On peut donc dire que le dropout aide à obtenir un modèle correct dans cette deuxième partie, mais il n'est pas le paramètre qui donne la meilleure performance finale.

Cette observation est intéressante pour comparer avec la première partie. Pour la base à 4 classes, le dropout n'était pas vraiment nécessaire, car les modèles simples donnaient déjà de très bons résultats. Pour la base à 32 classes, il devient plus utile, car le problème est plus difficile et le risque de surapprentissage est plus important.

2.4.4 Influence de l'optimiseur

Enfin, j'ai comparé l'optimiseur Adam avec RMSprop. Avec l'architecture A et 5 epochs, RMSprop donne une précision de validation de 0.7045 et une précision de test de 0.7271. Ces résultats sont meilleurs que certains essais, mais ils restent inférieurs à 75%.

Adam donne donc de meilleurs résultats dans cette étude, surtout avec 10 epochs. J'ai donc conservé Adam pour le modèle final, car il permet d'obtenir une précision supérieure à l'objectif demandé.

2.5 Choix du modèle final pour la base à 32 classes

Pour la base à 32 classes, j'ai choisi le modèle `P2_ep10_A_adam`. Ce modèle utilise l'architecture A, c'est-à-dire deux blocs convolutionnels, avec l'optimiseur Adam, un taux d'apprentissage de 0.001, une taille de batch de 32 et 10 epochs.

Ce choix est justifié par le fait que le modèle atteint une précision de validation de 0.8415 et une précision de test de 0.8250. Il dépasse donc l'objectif de 75%. De plus, ce modèle reste relativement simple, puisqu'il utilise seulement deux blocs convolutionnels. Les architectures plus complexes ne donnent pas de meilleurs résultats dans cette série d'expériences.

Le modèle final retenu pour cette deuxième partie est donc :

Paramètre	**Valeur**
Architecture	Variante A
Nombre de blocs convolutionnels	2
Optimiseur	Adam
Learning rate	0.001
Batch size	32
Epochs	10
Dropout	0.0
Best val accuracy	0.8415
Test accuracy	0.8250

Table 6 – Architecture finale retenue pour la base à 32 classes

2.6 Conclusion de la deuxième partie

Pour conclure cette deuxième partie, la classification avec 32 types de feuilles est plus difficile que celle avec 4 types. Les mêmes expériences donnent des résultats plus faibles que dans la première partie, ce qui montre que l'augmentation du nombre de classes complique la tâche du modèle.

Certaines configurations qui fonctionnaient très bien dans la première partie ne suffisent plus ici. Par exemple, avec 3 ou 5 epochs, l'architecture A n'atteint pas la précision demandée de 75%. En revanche, lorsque le nombre d'epochs est augmenté à 10, le modèle parvient à dépasser l'objectif.

Le modèle retenu est donc `P2_ep10_A_adam`, car il atteint une précision de test de 0.8250 tout en gardant une architecture simple. Cette comparaison montre qu'une architecture simple peut suffire pour une base réduite, mais qu'une base plus complexe demande un apprentissage plus long et une analyse plus attentive des résultats.

Conclusion générale

Ce projet a permis d'étudier la classification automatique de feuilles d'arbres à l'aide d'un réseau de neurones convolutionnel. Les deux parties ont été réalisées avec les mêmes expériences afin de comparer directement l'effet du nombre de classes sur les performances du modèle.

Pour la base à 4 classes, les résultats sont très élevés. Une architecture simple avec deux blocs convolutionnels suffit pour atteindre une précision de test de 1.0000. Cela montre que cette première base est relativement facile à apprendre pour le modèle, car le nombre de classes est faible et les feuilles à distinguer sont moins nombreuses.

Pour la base complète à 32 classes, les mêmes expériences donnent des résultats plus faibles. Les modèles avec 3 ou 5 epochs ne dépassent pas toujours l'objectif de 75%. Le meilleur résultat est obtenu avec l'architecture A et 10 epochs, avec une précision de test de 0.8250. Ce résultat reste satisfaisant, car il dépasse la précision demandée, mais il montre aussi que la classification devient plus difficile lorsque le nombre de classes augmente.

Base	**Nombre de classes**	**Modèle retenu**	**Test accuracy**
Partie 1	4	`P1_2blocs_A_adam`	1.0000
Partie 2	32	`P2_ep10_A_adam`	0.8250

Table 7 – Comparaison des modèles finaux retenus

La comparaison entre les deux parties montre donc qu'une architecture simple peut être suffisante pour une base réduite, mais qu'une base plus complexe demande un apprentissage plus long et une analyse plus fine des paramètres. L'étude confirme aussi l'intérêt de tester plusieurs configurations au lieu de choisir directement un seul modèle, car les performances peuvent varier fortement selon la taille et la difficulté de la base de données.